

Name:G.Eshwar

H.NO:2303A51808

BATCH:26

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
Instructor(s) Name		Mr. S Naresh Kumar	
		Ms. B. Swathi	
		Dr. Sasanko Shekhar Gantayat	
		Mr. Md Sallauddin	
		Dr. Mathivanan	
		Mr. Y Srikanth	
		Ms. N Shilpa	
		Dr. Rishabh Mittal (Coordinator)	
		Dr. R. Prashant Kumar	
		Mr. Ankushavali MD	
		Mr. B Viswanath	
		Ms. Sujitha Reddy	
		Ms. A. Anitha	
		Ms. M.Madhuri	
		Ms. Katherashala Swetha	
		Ms. Velpula sumalatha	
		Mr. Bingi Raju	
CourseCode	23CS002PC304	Course Title	AI Assisted Coding
Year/Sem	III/II	Regulation	R23
Date and Day of Assignment	Week10 – Friday	Time(s)	23CSBTB01 To 23CSBTB52
Duration	2 Hours	Applicable to Batches	All batches
Assignment Number:20.5(Present assignment number)/24(Total number of assignments)			

Q.No.	Question	Expected Time to complete
1	Lab 20 – Security Testing: Identifying Vulnerabilities in AI-Generated Code	Week10 - Friday

Lab Objectives:

- Understand how to test AI-generated code for common security vulnerabilities.
- Learn to apply secure coding principles while analyzing AI outputs.
- Practice detecting risks such as SQL injection, XSS, hardcoded credentials, and weak encryption.
- Enhance code reliability and safety by using AI for secure refactoring.

Task 1 – Password Storage Vulnerability Check

Task:

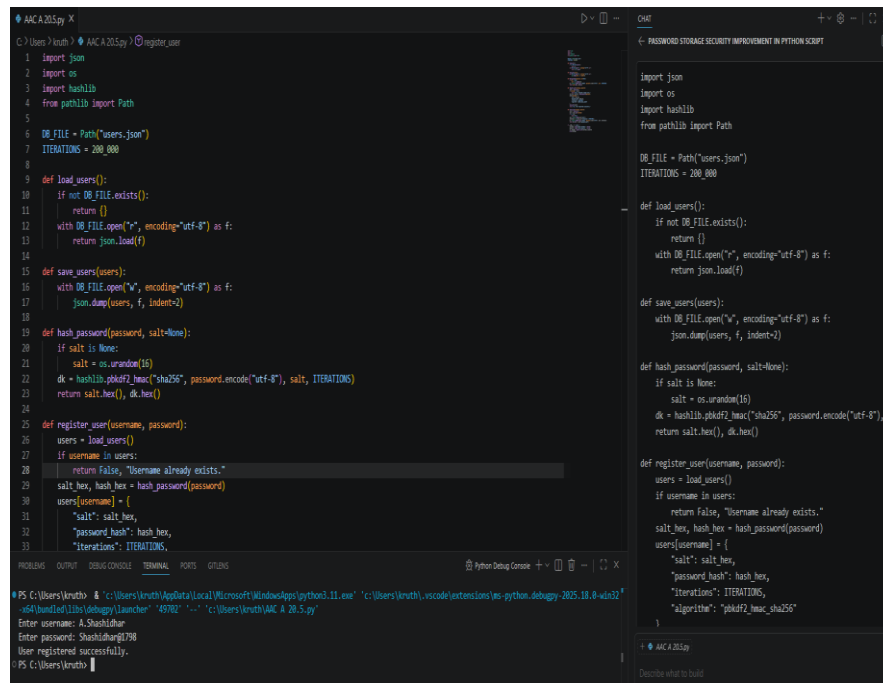
Analyze an AI-generated signup script for insecure password storage.

Instructions:

- Prompt AI to generate a Python user registration program.
- Check whether passwords are stored in plain text.
- Improve the script using hashing (bcrypt or hashlib).

Expected Output:

- A secure registration script with hashed password storage.



```
ACCA2023py X
C:\Users\kruth> python register_user
1 import json
2 import os
3 import hashlib
4 from pathlib import Path
5
6 DB_FILE = Path("users.json")
7 ITERATIONS = 200_000
8
9 def load_users():
10     if not DB_FILE.exists():
11         return []
12     with DB_FILE.open("r", encoding="utf-8") as f:
13         return json.load(f)
14
15 def save_users(users):
16     with DB_FILE.open("w", encoding="utf-8") as f:
17         json.dump(users, f, indent=2)
18
19 def hash_password(password, salt=None):
20     if salt is None:
21         salt = os.urandom(16)
22     dk = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, ITERATIONS)
23     return salt.hex(), dk.hex()
24
25 def register_user(username, password):
26     users = load_users()
27     if username in users:
28         return False, "Username already exists."
29     salt_hex, hash_hex = hash_password(password)
30     users[username] = {
31         "salt": salt_hex,
32         "password_hash": hash_hex,
33         "iterations": ITERATIONS,
34     }
35     save_users(users)
36     return True, "User registered successfully."
37
38 if __name__ == "__main__":
39     username = input("Enter username: ")
40     password = input("Enter password: ")
41     success, message = register_user(username, password)
42     print(message)
```

```
ACCA2023py X
PASSWORD STORAGE SECURITY IMPROVEMENT IN PYTHON SCRIPT
import json
import os
import hashlib
from pathlib import Path

DB_FILE = Path("users.json")
ITERATIONS = 200_000

def load_users():
    if not DB_FILE.exists():
        return []
    with DB_FILE.open("r", encoding="utf-8") as f:
        return json.load(f)

def save_users(users):
    with DB_FILE.open("w", encoding="utf-8") as f:
        json.dump(users, f, indent=2)

def hash_password(password, salt=None):
    if salt is None:
        salt = os.urandom(16)
    dk = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, ITERATIONS)
    return salt.hex(), dk.hex()

def register_user(username, password):
    users = load_users()
    if username in users:
        return False, "Username already exists."
    salt_hex, hash_hex = hash_password(password)
    users[username] = {
        "salt": salt_hex,
        "password_hash": hash_hex,
        "iterations": ITERATIONS,
        "algorithm": "pbkdf2_hmac_sha256"
    }
    save_users(users)
    return True, "User registered successfully."

if __name__ == "__main__":
    username = input("Enter username: ")
    password = input("Enter password: ")
    success, message = register_user(username, password)
    print(message)
```

```
PS C:\Users\kruth> python register_user
Enter username: A.Shashidhar
Enter password: Shashidhar@123
User registered successfully.
PS C:\Users\kruth>
```

```
def register_user(username, password):
    users = load_users()
    if username in users:
        return False, "Username already exists."
    salt_hex, hash_hex = hash_password(password)
    users[username] = {
        "salt": salt_hex,
        "password_hash": hash_hex,
        "iterations": ITERATIONS,
        "algorithm": "pbkdf2_hmac_sha256"
    }
    save_users(users)
    return True, "User registered successfully."

def verify_user(username, password):
    users = load_users()
    user = users.get(username)
    if not user:
        return False
    salt = bytes.fromhex(user["salt"])
    iterations = int(user.get("iterations", ITERATIONS))
    dk = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, iterations)
    return dk.hex() == user["password_hash"]

if __name__ == "__main__":
    username = input("Enter username: ").strip()
    password = input("Enter password: ").strip()
    dk, message = register_user(username, password)
    print(message)
```

```
PS C:\Users\kruth > python .\AKA_20.5.py
Enter username: A.Shahidur
Enter password: Shaahidur788
User registered successfully.
```

Task 2 – Hardcoded Secrets Detection

Task:

Evaluate AI-generated code for hardcoded API keys or credentials.

Instructions:

- Ask AI to generate a script that connects to an external API.
- Identify if API keys/passwords are written directly in code.
- Refactor using environment variables or config files.

Expected Output:

- A secure script with secrets managed properly.

```
def get_weather(city: str) -> dict:
    api_key = load_api_key()
    if not api_key:
        raise ValueError("Missing WEATHER_API_KEY in environment, weather_config.json, and runtime input")
    params = {"q": city, "appid": api_key, "units": "metric"}
    response = requests.get(BASE_URL, params=params, timeout=15)
    if response.status_code == 401:
        raise RuntimeError("Invalid WEATHER_API_KEY. Please verify or generate a new key.")
    response.raise_for_status()
    return response.json()
```

```
PS C:\Users\kruth > python .\AKA_20.5.py
requests.exceptions.HTTPError: 401 (Client Error: Unauthorized for url: https://api.openweathermap.org/data/2.5/weather?q=Parkal&apiId=60163d20913ac61b6d9b0a20640a15a20a)
C:\Users\kruth> python .\AKA_20.5.py
Enter WEATHER_API_KEY: 84951c20961b6d9b0a20640a15a20a
Invalid WEATHER_API_KEY. Please verify or generate a new key.
```

```
AAC A 20.5.py X
C:\Users\kruth> AAC A 20.5.py > get_weather
21 def get_weather(city: str) -> dict:
22     api_key = load_api_key()
23     if not api_key:
24         api_key = input("Enter WEATHER_API_KEY: ").strip()
25     if not api_key:
26         raise RuntimeError("Missing WEATHER_API_KEY in environment, weather_config.json, and runtime input")
27     params = {"q": city, "appid": api_key, "units": "metric"}
28     response = requests.get(BASE_URL, params=params, timeout=15)
29     if response.status_code == 401:
30         raise RuntimeError("Invalid WEATHER_API_KEY. Please verify or generate a new key.")
31     response.raise_for_status()
32     return response.json()
33
34 if __name__ == "__main__":
35     try:
36         city = input("City: ").strip()
37         data = get_weather(city)
38         print(data)
39     except requests.exceptions.RequestException as e:
40         print(f"Network/API error: {e}")
41     except RuntimeError as e:
42         print(e)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
PS C:\Users\kruth> & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '55964' '-.' 'c:\Users\kruth\AAC A 20.5.py'
raise HTTPError(http_error_msg, response=self)
requests.exceptions.HTTPError: 401 Client Error: Unauthorized for url: https://api.openweathermap.org/data/2.5/weather?q=Parkal&appid=84f61c6269619ae6348e093b0aa4209a&units=metric
PS C:\Users\kruth> c; cd 'c:\Users\kruth'; & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\extensions\ms-python.d
ebugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '50806' '-.' 'c:\Users\kruth\AAC A 20.5.py'
City: Parkal
Enter WEATHER_API_KEY: 84f61c6269619ae6348e093b0aa4209a
Invalid WEATHER_API_KEY. Please verify or generate a new key.
```

Task 3 – File Upload Security Validation

Task:

Test an AI-generated file upload program for unsafe file handling.

Instructions:

- Prompt AI to generate a Flask file upload example.
- Check for vulnerabilities like uploading executable files.
- Add validation for file type, size, and secure filename handling.

Expected Output:

- A secure file upload implementation.

```

AAC A 20.5.py 2 X
C:\Users\kruth > AAC A 20.5.py ...
1 from flask import Flask, request, jsonify
2 from werkzeug.utils import secure_filename
3 from pathlib import Path
4 import uuid
5
6 app = Flask(__name__)
7 app.config["MAX_CONTENT_LENGTH"] = 5 * 1024 * 1024
8
9 UPLOAD_DIR = Path("uploads")
10 UPLOAD_DIR.mkdir(exist_ok=True)
11
12 ALLOWED_EXTENSIONS = ["png", "jpg", "jpeg", "pdf", "text"]
13 EXTENSION_TO_KIND = {
14     "png": ("img"),
15     "jpg": ("img"),
16     "jpeg": ("img"),
17     "pdf": ("pdf"),
18     "text": ("text"),
19 }
20
21 def detect_kind(data: bytes) -> str:
22     if data.startswith(b"\xFF\xD8"):
23         return "img"
24     if data.startswith(b"\xFF\xE0"):
25         return "img"
26     if data.startswith(b"PK"):
27         return "zip"
28     if data.startswith(b"%PDF"):
29         return "pdf"
30     if data.startswith(b"0x00000000"):
31         return "text"
32     if "L\x00" in data:
33         return "binary"
34     return "unknown"
35
36 PS C:\Users\kruth> cd 'c:\Users\kruth' & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\extensions\ms-
ebugpy-2025.18.0-win32-x64\buggy\launcher' 'AAC A 20.5.py'
ModuleNotFoundError: No module named 'flask'
PS C:\Users\kruth> cd 'c:\Users\kruth' & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\extensions\ms-
ebugpy-2025.18.0-win32-x64\buggy\launcher' 'AAC A 20.5.py'
* Serving Flask app 'AAC A 20.5'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit

```

```

AAC A 20.5.py 2 X
C:\Users\kruth > AAC A 20.5.py ...
21 def detect_kind(data: bytes) -> str:
22     try:
23         data.decode("utf-8")
24         return "txt"
25     except UnicodeDecodeError:
26         return "unknown"
27
28 @app.post("/upload")
29 def upload_file():
30     if "file" not in request.files:
31         return jsonify({"error": "No file part"}), 400
32
33     file = request.files["file"]
34     if file.filename is None or file.filename.strip() == "":
35         return jsonify({"error": "No selected file"}), 400
36
37     original_name = secure_filename(file.filename)
38     if original_name == "":
39         return jsonify({"error": "Invalid filename"}), 400
40
41     ext = Path(original_name).suffix.lower().lstrip(".")
42     if ext not in ALLOWED_EXTENSIONS:
43         return jsonify({"error": "File extension not allowed"}), 400
44
45     head = file.stream.read(8192)
46     if not head:
47         return jsonify({"error": "Empty file"}), 400
48
49     kind = detect_kind(head)
50     if kind in ("exe", "zip", "binary", "unknown"):
51         return jsonify({"error": "Unsafe or unsupported file content"}), 400
52
53     if kind not in EXTENSION_TO_KIND[ext]:
54
55 PS C:\Users\kruth> cd 'c:\Users\kruth' & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\extensions\ms-
ebugpy-2025.18.0-win32-x64\buggy\launcher' '57474' '--' 'c:\Users\kruth\AAC A 20.5.py'
ModuleNotFoundError: No module named 'flask'
PS C:\Users\kruth> cd 'c:\Users\kruth' & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\extensions\ms-
ebugpy-2025.18.0-win32-x64\buggy\launcher' '57510' '--' 'c:\Users\kruth\AAC A 20.5.py'
* Serving Flask app 'AAC A 20.5'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit

```

```
AAC A 20.5.py 2 X
C:\Users\kruth > AAC A 20.5.py > ...
41 def upload_file():
50     if original_name == "":
51         return jsonify({"error": "Invalid filename"}), 400
52
53     ext = Path(original_name).suffix.lower().lstrip(".")
54     if ext not in ALLOWED_EXTENSIONS:
55         return jsonify({"error": "File extension not allowed"}), 400
56
57     head = file.stream.read(8192)
58     if not head:
59         return jsonify({"error": "Empty file"}), 400
60
61     kind = detect_kind(head)
62     if kind in ("exe", "zip", "binary", "unknown"):
63         return jsonify({"error": "Unsafe or unsupported file content"}), 400
64
65     if kind not in EXTENSION_TO_KIND[ext]:
66         return jsonify({"error": "File content does not match extension"}), 400
67
68     file.stream.seek(0)
69     stored_name = f"{uuid.uuid4().hex}_{original_name}"
70     destination = UPLOAD_DIR / stored_name
71     file.save(destination)
72
73     return jsonify({"message": "Upload successful", "file": stored_name}), 201
74
75 @app.errorhandler(413)
76 def file_too_large():
77     return jsonify({"error": "File too large. Max size is 5 MB"}), 413
78
79 if __name__ == "__main__":
80     app.run(debug=False)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

PS C:\Users\kruth> cd 'c:\Users\kruth'; & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\ebugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '57474' '--' 'c:\Users\kruth\AAC A 20.5.py'
ModuleNotFoundError: No module named 'flask'
PS C:\Users\kruth> cd 'c:\Users\kruth'; & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\ebugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '57510' '--' 'c:\Users\kruth\AAC A 20.5.py'
* Serving Flask app 'AAC A 20.5'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
PS C:\Users\kruth>
```

Task 4 – Command Injection Prevention

Task:

Analyze an AI-generated Python script that executes system commands.

Instructions:

- Ask AI to generate code using `os.system()` or `subprocess`.
- Identify command injection risks with user input.
- Fix using safe subprocess calls and input validation.

Expected Output:

- A secure command execution script.

```
AAC A 20.5.py 2 X
C:\Users\kruth > AAC A 20.5.py > ...
1 import re
2 import subprocess
3
4 ALLOWED_COMMANDS = [
5     "ping", "whoami", ["cat", "n"], "time", ["cat", "n"]],
6     "netcat", ["whoami", "n"], "time", []],
7 ]
8 SAFE_TARGET_PATTERN = re.compile(r"^[a-zA-Z0-9._]{1,255}$")
9
10 def validate_target(target: str) -> str:
11     value = target.strip()
12     if not SAFE_TARGET_PATTERN.fullmatch(value):
13         raise ValueError("Invalid target")
14     if value.startswith(".") or value.endswith(".") or ". " in value:
15         raise ValueError("Invalid target")
16     return value
17
18 def validate_count(count: str) -> int:
19     value = int(count)
20     if value < 1 or value > 10:
21         raise ValueError("Count must be between 1 and 10")
22     return value
23
24 def run_safe_command(target: str, count: int = 4) -> str:
25     cmd = command.strip().lstrip()
26     if cmd not in ALLOWED_COMMANDS:
27         raise ValueError("Unsupported command")
28     safe_target = validate_target(target)
29     safe_count = validate_count(count)
30     if cmd == "ping":
31         args = ["ping", "-n", str(safe_count), safe_target]
32     else:
33         args = ["netcat", safe_target]
34
35     result = subprocess.run(args, capture_output=True, text=True)
36     return result.stdout
37
38 if __name__ == "__main__":
39     target = input("Enter target IP or domain: ")
40     count = input("Enter count (1-10): ")
41     result = run_safe_command(target, count)
42     print(result)
43
44 PS C:\Users\kruth> cd 'c:\Users\kruth'; & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\ebugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '57510' '--' 'c:\Users\kruth\AAC A 20.5.py'
Command (ping) arguments: ping
Enter count (1-10, default: 4): 4
Pinging google.com [142.250.182.78] with 32 bytes of data:
Reply from 142.250.182.78: bytes=32 time=28ms TTL=116
Reply from 142.250.182.78: bytes=32 time=28ms TTL=116
Reply from 142.250.182.78: bytes=32 time=28ms TTL=116
Reply from 142.250.182.78: bytes=32 time=28ms TTL=116

```



```
MCA205py 1 X
1 from flask import Flask, request, render_template_string, redirect, url_for, session, abort
2 import re
3 import secrets
4
5 app = Flask(__name__)
6 app.config["SECRET_KEY"] = secrets.token_hex(16)
7 app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///"
8
9 DB_URL_RE = re.compile(r"^(?!(?:[a-z0-9]+(?:[a-z0-9-+]{0,63}[a-z0-9])?\.(?:[a-z0-9]+(?:[a-z0-9-+]{0,63}[a-z0-9])?)*)$")
10 NAME_RE = re.compile(r"^(?!(?:[a-z0-9-+]{0,63}[a-z0-9])?\.(?:[a-z0-9]+(?:[a-z0-9-+]{0,63}[a-z0-9])?)*)$")
11
12 feedback_store = []
13
14 page = """
15 <!doctype html>
16 <html lang=en>
17 <head>
18 <meta charset="utf-8">
19 <meta name="viewport" content="width=device-width,initial-scale=1">
20 <title>Secure Feedback Form</title>
21 <style>
22 body { font-family: Segoe UI, sans serif; max-width: 768px; margin: 32px auto; padding: 0 16px; }
23 form { display: grid; gap: 16px; }
24 input, textarea { width: 100%; padding: 16px; }
25 button { padding: 16px 16px; width: 100%; text-align: center; }
26 <div class="error">{{ error }}</div>
27 <div class="success">{{ success }}</div>
28 <div class="name">{{ name }}</div>
29 <div class="email">{{ email }}</div>
30 <div class="message">{{ message }}</div>
31 </style>
32 </head>
33 <body>
34 <div class="header">
35 <div class="error">{{ error }}</div>
36 <div class="success">{{ success }}</div>
37 <div class="name">{{ name }}</div>
38 <div class="email">{{ email }}</div>
39 <div class="message">{{ message }}</div>
40 <div class="submit">
41 <button type="submit">Submit</button>
42 </div>
43 </body>
44 </html>
45 """
46
47 @app.route("/")
48 def index():
49     return render_template_string(page)
50
51 @app.route("/feedback")
52 def feedback():
53     if request.method == "POST":
54         error = None
55         success = None
56         name = request.form.get("name")
57         email = request.form.get("email")
58         message = request.form.get("message")
59
60         if not name:
61             error = "Name is required"
62         elif not email:
63             error = "Email is required"
64         elif not message:
65             error = "Message is required"
66         elif not DB_URL_RE.match(email):
67             error = "Invalid email address"
68         elif not NAME_RE.match(name):
69             error = "Invalid name"
70
71         if error:
72             return render_template_string(page)
73
74         feedback_store.append({
75             "name": name,
76             "email": email,
77             "message": message,
78             "timestamp": datetime.now().isoformat()
79         })
80
81         success = "Feedback submitted successfully"
82
83         return render_template_string(page)
84
85     return render_template_string(page)
```

AAC A.20.5.py 1Secure Feedback Form X

→ http://127.0.0.1:5000/feedback

Feedback

Name

Kruthan Kiran

Email

kruthankiran@gmail.com

Message

Hello There its Kruthan (2303A51404)

Submit

Recent Feedback

- No feedback yet.

→ http://127.0.0.1:5000/feedback

Feedback

Feedback submitted successfully.

Name

Email

Message

Submit

Recent Feedback

- Kruthan Kiran** (kruthankiran@gmail.com): Hello There its Kruthan (2303A51404)

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.